

初赛模拟卷15

一、单项选择题（共15题，每题2分，共计30分;每题有且仅有一个正确选项）

1. 一棵完全二叉树的结点总数为 41，其叶结点数为()。
A.18个
B.20个
C.21个
D.19个
2. C++ 程序运行时，是在哪个存储器上进行的?()
A.RAM
B.ROM
C.CACHE
D.硬盘
3. 下列不同进制的数中，最大的一个数是()。
A.八进制数 $(334.1)_8$
B.二进制数 $(11011011)_2$
C.十进制数 $(220.1)_{10}$
D.十六进制数 $(DC.1)_{16}$
4. 一棵树 T 有 2 个度数为 2 的结点、有 1 个度数为 3 的结点、有 3 个度数为 4 的结点，那么树 T 有()个树叶。
A.18
B.14
C.6
D.7
5. 定义一种字符串操作，一次可以将其中一个元素移到任意位置。举例说明，对于字符串“BCA”可以将“A”移到“B”之前，变字符串“ABC”，如果要将字符串“DACHEBGIF”变成

“ABCDEFGH” 最少需要()次操作。

- A.6
- B.5
- C.4
- D.7

6. 计划展出 10 幅不同的画, 其中 1 幅水彩画、4 幅油画、5 幅国画, 排成一行陈列, 要求同一品种的画必须连在一起, 并且水彩画不放在两端, 那么不同的陈列方式有()种。

- A.5760
- B.2880
- C.17280
- D.8640

7. 在高速缓存系统中, 主存容量为 $12MB$, Cache 容量为 $400KB$, 则该存储系统的容量为()

- A. $12MB + 400KB$
- B. $12MB$
- C. $12MB - 12MB + 400KB$
- D. $12MB - 400KB$

8. 在非空的双向链表中由 q 所指向的结点右边插入一个由 p 所指向的结点, 其动作依次为:

$p \rightarrow \text{left} = q; p \rightarrow \text{right} = q \rightarrow \text{right}; q \rightarrow \text{right} = p; ()$ 。

- A. $q \rightarrow \text{left} = p;$
- B. $q \rightarrow \text{right} \rightarrow \text{left} = p;$
- C. $p \rightarrow \text{right} \rightarrow \text{left} = p;$
- D. $p \rightarrow \text{left} \rightarrow \text{left} = p;$

9. 一个自然数在十进制下有 n 位, 则它在二进制下的位数与() 最接近。

- A. $5n$
- B. $n * \log_2 10$
- C. $10 * \log_2 n$
- D. $10^n \log_2 n$

10. 十进制表达式 $(3 * 512 + 7 * 64 + 4 * 8 + 3) + 1$ 的结果以二进制形式表示为 ()。

- A. $(10111100101)_2$
- B. $(11111100100)_2$
- C. $(111110100011)_2$
- D. $(11111101101)_2$

11. 以下关于个人计算机操作系统的说法正确的是 ()。

- A. Bill Gates, 他创办的 Microsoft 公司开发了 Linux 系列系统。
- B. Steve Jobs, 他的苹果公司开发了 IOS 系统。
- C. 目前 Linux 和 Windows 系统都是开源的, 网上都能搜索并下载。
- D. Unix 不是操作系统。

12. 一篇文章有 200 个中文字符, 50 个英文字符, 20 个半角空格, 如果只考虑存储这些文字本身需要()个字节。
- A. 470 字节
 - B. 270 字节
 - C. 400 字节
 - D. 320 字节
13. 计算机直接识别和执行的语言是 () 。
- A.C++
 - B.汇编语言
 - C.高级语言
 - D.机器语言
14. 现代普通计算机的网关不能设置为()。
- A. 168.10.7.100
 - B. 10.16.13.100
 - C. 234.123.6.5
 - D. 127.0.0.1
15. 假设我们用 $d=(a_1,a_2,\dots,a_5)$, 表示无向图 G 的 5 个顶点的度数, 下面给出的哪组 d 值合理 () 。
- A.{1, 2, 2, 1, 1}
 - B.{5, 4, 3, 2, 1}
 - C.{2, 2, 2, 2, 2}
 - D.{3, 3, 3, 2, 2}

二、阅读程序 (程序输入不超过数组或字符串定义的范围;其中判断题1.5分, 第5题分别为3,3,4分, 第6题分别为4,4,4分, 共计40分)

第一题

```
01 #include <iostream>
02 using namespace std;
03 int main() {
04     const int SIZE = 100;
05     int height[SIZE], num[SIZE], n, ans;
06     cin >> n;
07     for (int i = 0; i < n; i++) {
08         cin >> height[i];
09         num[i] = 1;
```

```

10         for (int j = 0; j < i; j++) {
11             if ((height[j] < height[i]) && (num[j] >= num[i]))
12                 num[i] = num[j] + 1;
13         }
14     }
15     ans = 0;
16     for (int i = 0; i < n; i++) {
17         if (num[i] > ans) ans = num[i];
18     }
19     cout << ans << endl;
20     return 0;
21 }

```

1. n 最大可以为 99 才能保证程序不会出错误。()
 - A.正确
 - B.错误
2. 程序功能：输入长度为 n 的整数序列，求最长上升子序列的长度。()
 - A.正确
 - B.错误
3. 若将第 10 行的 $j < i$ 改为 $j \leq i$ ，程序运行时会发生错误。()
 - A.正确
 - B.错误
4. 本程序的时间复杂度为 $O(n^2)$ 。()
 - A.正确
 - B.错误
5. 若输入

6
2 5 3 11 12 4

则输出为()

- A.3
- B.4
- C.2
- D.5

6. 若输入

5
2 3 4 5 6

则第 12 行 $\text{num}[i] = \text{num}[j] + 1;$ 执行()次。

- A.6

- B.8
- C.10
- D.12

第二题

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int mod = 2048;
04 long long c, n;
05 long long kasumi(long long x, long long mi) {
06     long long res = 1;
07     while (mi) {
08         if (mi & 1) {
09             res = (res * x) % mod;
10         }
11         x = (x * x) % mod;
12         mi >>= 1;
13     }
14     return res;
15 }
16 int main() {
17     cin >> n >> c;
18     if (n == 3) {
19         printf("%lld", c * (c - 1));
20         return 0;
21     }
22     long long ans = ((kasumi(c - 1, n) + (c - 1) * kasumi(-1, n))
23                     % mod + mod) % mod;
24     cout << ans;
25     return 0;
26 }
```

1. 将第 9 行 `res = (res * x) % mod;` 和第 11 行 `x = (x * x) % mod;` 的括号去掉, 程序输出结果一定不变。 ()
A.正确
B.错误
2. 将第 12 行的 `mi >>= 1` 改为 `mi /= 2`, 程序输出结果一定不变。 ()
A.正确
B.错误
3. 若输入为 4 4, 则输出为 78。 ()
A.正确

B.错误

4. 此程序的时间复杂度为 $O(\log n)$ 。

A.正确

B.错误

5. 若输入为 3 4，则输出为（）。

A.18

B.19

C.12

D.8

6. kasumi(2046, 13) 的返回值为（）。

A.2024

B.2

C.0

D.2022

第三题

```
01 #include<stdio>
02 int n, r, num[10000];
03 bool mark[10000];
04 void print() {
05     for (int i = 1; i <= r; i++)
06         printf("%d ", num[i]);
07     printf("\n");
08 }
09 void search(int x) {
10     for (int i = 1; i <= n; i++)
11         if (!mark[i]) {
12             num[x] = i;
13             mark[i] = true;
14             if (x == r) print();
15             search(x + 1);
16             mark[i] = false;
17         }
18 }
19 int main() {
20     scanf("%d%d", &n, &r);
21     search(1);
22     return 0;
23 }
```

1. 若 $n < r$, 则程序无输出。 ()
A.正确
B.错误
2. 程序结束时, 对任意 $1 \leq i \leq n$, $\text{mark}[i] == 0$ 。 ()
A.正确
B.错误
3. 此程序的时间复杂度为 $O(n)$ 。 ()
A.正确
B.错误
4. 若输入为 4 3 , 则输出中数字 1 和 2 的个数不同。 ()
A.正确
B.错误
5. 若输入为 6 3 , 则函数 `print` 的执行次数为 () 。
A.60
B.120
C.6
D.720
6. 若输入为 7 4 , 则输出的最后一行为 () 。
A.1 2 3 4
B.4 5 6 7
C.4 3 2 1
D.7 6 5 4

三、完善程序 (单选题, 每题 3 分, 共计 30 分)

第一题

给定一个字符串 S , 有 q 组询问, 每次给定一个字符串 T , 求字符串 T 是否是 S 中的一个子序列。

数据保证 $1 \leq |S| \leq 10^5, 1 \leq \sum |T| \leq 10^6$, 所有字符串仅包含小写字母。

```
#include <iostream>
using namespace std;

const int BASE = 26;

string S, T;
int q, Pos[100005][BASE];
```

```

int main(){
    cin >> S;
    int n = S.size();
    for (int i = 0; i < BASE; i++)
        ___(1)___ = -1;
    for (int i = n - 1; i >= 0; i--){
        for (int j = 0; j < BASE; j++)
            ___(2)___;
        ___(3)___;
    }
    cin >> q;
    while (q--){
        cin >> T;
        int len = T.size(), now = 0;
        for (int i = 0; ___(4)___; i++)
            now = ___(5)___;
        if (now != -1)
            cout << "YES\n";
        else
            cout << "NO\n";
    }
    return 0;
}

```

1. (1) 处应填 ()

- A. Pos[i][n - 1]
- B. Pos[i][n]
- C. Pos[n - 1][i]
- D. Pos[n][i]

2. (2) 处应填 ()

- A. Pos[i][j] = Pos[i - 1][j]
- B. Pos[i][j] = Pos[i + 1][j]
- C. Pos[i][j] = Pos[i][j - 1]
- D. Pos[i][j] = Pos[i][j + 1]

3. (3) 处应填 ()

- A. Pos[i][S[i] - 'a'] = i
- B. Pos[i][S[i] - 'A'] = i
- C. Pos[i][S[i]] = i
- D. Pos[i][i] = S[i]

4. (4) 处应填 ()

- A. i < len

- B. now != -1
- C. i < len && now != n
- D. i < len && now != -1

5.(5) 处应填 ()

- A. Pos[now][T[i] - 'a']
- B. Pos[now][S[i] - 'a']
- C. Pos[now + 1][T[i] - 'a']
- D. Pos[now + 1][S[i] - 'a']

第二题

(排列数)输入两个正整数 $n, m (1 < n < 20, 1 < m < n)$, 在 $1 \sim n$ 中任取 m 个数,按字典序从小到大输出所有这样的排列。

例如: 输入: 3 2 输出:

```
1 2
1 3
2 1
2 3
3 1
3 2
```

```
#include <iostream>
#include <cstring>
using namespace std;

const int SIZE = 25;
bool used[SIZE];
int data[SIZE];
int n, m, i, j, k;
bool flag;

int main() {
    cin >> n >> m;
    memset(used, false, sizeof(used));
    for (i = 1; i <= m; i++) {
        data[i] = i;
        used[i] = true;
    }
    flag = true;
    while (flag) {
```

```

for (i = 1; i <= m - 1; i++) cout << data[i] << " ";
cout << data[m] << endl;
flag = ___(1)___;
for (i = m; i >= 1; i--) {
    ___(2)___;
    for (j = data[i] + 1; j <= n; j++)
        if (!used[j]) {
            used[j] = true;
            data[i] = ___(3)___;
            flag = true;
            break;
        }
    if (flag) {
        for (k = i + 1; k <= m; k++)
            for (j = 1; j <= ___(4)___; j++)
                if (!used[j]) {
                    data[k] = j;
                    used[j] = true;
                    break;
                }
            ___(5)___;
    }
}
return 0;
}

```

1. (1) 处应填()

- A. false
- B. true
- C. 1
- D. -1

2. (2) 处应填()

- A. used[i] = true
- B. data[i] = i
- C. used[data[i]] = true
- D. used[data[i]] = false

3. (3) 处应填()

- A. j
- B. i
- C. true
- D. false

4. (4) 处应填()

A. n

B. m

C. i

D. j

5. (5) 处应填()

A. return 0

B. exit

C. continue

D. break